

AI-Powered Auto-Evaluation Tool

Wireframe - Q&A

To better serve our product users, we are planning two unique login experiences.

- **Candidates** who will be using an AI bot as a learning tool, answering its questions to acquire new knowledge.
- **Admins**, allowing them to oversee what candidates have learned and determine how those acquired concepts can be practically applied within the company.

Here we've developed a simple wireframe targeting a typical exchange between the Candidate and the AI.

Example : AOP- Candidate and AI Exchange

AI: Hey there! I'm here to help us figure out how well you understand certain ideas. We'll do this by just chatting about a topic. Before we start, can you please enter your name?

Candidate: Aathithyan K K

AI: That's great Aathithyan!! To get us rolling, what subject are you feeling up for today? We could talk about:

- **Aspect-Oriented Programming (AOP)** – a pretty powerful way to build software.
- The **challenges of real-time data processing** in IoT devices – super relevant right now, for sure.
- Or maybe the key parts of a **scalable microservices architecture**.

Candidate: I would like to go with AOP which I have heard about but not completely clear about.

AI: Great!! Sure. Let's get rolling. Let's start with this: When you hear the word Aspect-Oriented Programming what pops in your head?

Candidate: As far as I know it is related to separating concerns. Basically separating the code into chunks so that the main code base doesn't become very messy.

AI: That's a very clear and accurate starting point! You've hit on the core idea: "separating concerns" to prevent the main codebase from becoming "messy."

Could you elaborate a bit on why this "messiness" is a problem? What specific issues arise in traditional programming when these concerns aren't separated, and how does AOP aim to mitigate those?

Candidate: I am not sure, how it doesn't happen in traditional programming can you give me a clue?

AI: Certainly, let's think about a common scenario.

Imagine you have many different parts of your application (e.g., various functions, methods, or modules) that all need to perform the same task, like logging every time a specific operation happens, or checking security permissions before allowing access.

In traditional programming, where would you typically put that logging or security code? How would it appear across your codebase?

Candidate: I would place it under a separate logging script I feel?

AI: That's a good thought for organizing your logging logic itself into a separate script or module. Many developers do that to keep the logging implementation clean.

However, consider where you'd actually call or invoke that logging logic. If you want to log every time a user logs in, or every time a database record is updated, where would the `logThisEvent()` or `checkPermissions()` calls appear in your main application code? Would they be confined to just one place, or would they be spread out?

Candidate: I understand that they would be spread out.

AI: You're absolutely right, they would be spread out!

This "spreading out" of code related to a single concern (like logging or security) across many different modules or functions is precisely the "messiness" or problem that AOP tries to address. This scattered code is often referred to as a cross-cutting concern.

Can you think of any *other* examples of concerns, besides logging and security, that might typically be scattered across different parts of an application and could therefore be considered "cross-cutting"?

Candidate: Can I say caching?

Basically to improve performance, the logic checks if the data is in cache - retrieves it and stores it after computation.

This can happen in multiple places in the code.

AI: Absolutely, caching is an *excellent* example!

You've perfectly articulated why it's a cross-cutting concern: the logic for checking, retrieving from, and storing data in a cache needs to be applied in various places throughout your

application where performance optimization is desired. This often leads to the same caching logic being duplicated or scattered.

So, to summarize, AOP provides a way to modularize these cross-cutting concerns (like logging, security, and caching). What do you call the specific construct or modular unit in AOP that encapsulates these concerns? It's the "thing" that holds the logging logic, or the caching logic, separately.

Candidate: Is it the Aspect?

AI: You got it! Precisely.

The modular unit that encapsulates a cross-cutting concern in AOP is called an Aspect. Now that we've established what an aspect is and what problems it solves, let's talk about how aspects actually interact with your main code. An aspect doesn't directly modify the original source code. Instead, it "injects" its behavior at specific points.

What are these specific points in the execution of a program where an aspect can insert its behavior called in AOP terminology?

Candidate: Are Join Points are the specific points where an aspect can insert its behaviour?

But I am not able to understand exactly how it's put under use? Can you give me an example to understand?

AI: You're right, Join Points are specific points in a program's execution where an aspect's behavior can be inserted.

Using your caching example: For a `getProductById(productId)` method that fetches data, the "join point" would be the execution of this method itself. Other join points could be method calls, field settings, exception throws, or object initialization.

The caching aspect's behavior would be: "Before `getProductById` executes, check the cache. If data is found, return it. If not, proceed with `getProductById`, then store the result in the cache."

Essentially, a Join Point is an opportunity for an aspect to "hook" into the program's flow.

Candidate: It makes sense now. We can move forward!

AI: Fantastic! I'm glad that example made it clearer.

Now that we understand Join Points as the specific opportunities or hooks for an aspect to act, we need a way for an aspect to *select* which of these many potential join points it's interested in. An aspect usually doesn't want to apply its logic at *every single* method call or field access in the entire application.

What is the term in AOP that refers to a set of one or more join points where an aspect's behavior should be applied? It's like a query or an expression that filters down the universe of all possible join points.

Candidate: I am not sure of the term. But I feel they might just be expressions that determine whether a given join point matches a certain criteria.

For example: A criteria can be based on thing like with Class names

AI: You're very, very close to the exact term, and your description of its function is spot-on! You've perfectly described what it *does*.

The term you're looking for, which represents those "expressions that determine whether a given join point matches a certain criteria" (like class names, method names, etc.), is a Pointcut.

So, a Pointcut is essentially a predicate or an expression that matches specific join points. It defines *where* the aspect's advice should be executed.

Now, we have Join Points (the where/when) and Pointcuts (the selector). What do we call the actual code or behavior that an aspect executes *at* a join point, which has been matched by a pointcut? For instance, the actual caching logic or logging statement that runs when a method is called.

Candidate: Is the term Advice? I think we can end it here. Can you please summarise what we have learnt till now?

AI: You've hit it precisely!

The actual code or behavior that an aspect executes at a matched join point is called Advice.

So, to recap these crucial terms:

- A Join Point is a specific, executable point in your program (e.g., a method call, an exception being thrown).
- A Pointcut is an expression that selects a set of join points where an aspect should act.
- Advice is the code an aspect executes at those join points (e.g., the logging code, the caching logic).
- An Aspect itself is the modular unit that brings together pointcuts and advice, encapsulating a cross-cutting concern.

Candidate: Thanks! Can you give me sources and tips for me to learn and improve further on this topic?

AI: *Gives the inputs.....*

Once a conversation ends, the **Admin receives an evaluation report**. This report quantifies the candidate's understanding of the concept and suggests how their knowledge could benefit the company.